

PHast-R: Bit-Parallel Perfect Hashing with Patterns and Cuckoo Repair

Abstract

PHast and its fast-to-build variant PHast⁺ are minimal perfect hash functions with the fastest known queries: a key is mapped to a bucket by a linear function of its hash, and a fixed-width per-bucket seed selects the positions of the keys of the bucket inside small overlapping slices of the output range. PHast⁺ uses additive placement, so that feasible seeds can be found with bit-parallel operations, but it needs about 0.2 more bits per key than PHast. We analyze where this space goes: with a minimal output range, every bumped key leaves a hole that must be encoded, and the rigidity of additive placement both forces more bumping and causes *self-collisions* that no seed can resolve. We then propose PHast-R, in which the seed selects one of R independent offset patterns in addition to a shift, and in which construction *repairs* failing buckets by cuckoo-style eviction, detecting with a bit-parallel half adder the shifts at which exactly one key is blocked. Queries perform the same operations as in PHast⁺, plus a shift and a mask. With 8-bit seeds, PHast-R matches the space of PHast (1.92 bits per key) while building 4.4 times faster; with 10-bit seeds it reaches 1.85 bits per key, the space of PHast, 12 times faster, and improves on all PHast⁺ variants at the same construction time.

1 Introduction

A *minimal perfect hash function* (MPHF) maps a static set of n keys bijectively to $[0..n)$; it requires at least $\log_2 e \approx 1.443$ bits per key. Among the many practical constructions, *bucket placement* gives the fastest queries: keys are hashed to buckets, and a per-bucket seed chooses positions for the keys of the bucket so that no two keys collide.

PHast [?] pushes this approach to its limit. A key with hash c goes to bucket $\lfloor cB/2^{64} \rfloor$; the seed s of the bucket, stored in a plain array of S -bit values, places the key at $\lfloor c(m-L+1)/2^{64} \rfloor + p(s, c)$, that is, inside a *slice* of length L whose start grows linearly with c . Buckets for which no seed works are *bumped* (seed 0) and handled by further levels; an Elias–Fano sequence [?, ?] maps their outputs to the free slots of the first level. A query thus reads a single seed for almost all keys. Seeds are assigned by a sweep over the buckets that keeps a small window of buckets in a priority queue, favoring large buckets, and chooses for each bucket the feasible seed that minimizes the sum of the positions of its keys.

In PHast, $p(s, c)$ is a pseudorandom function, so finding the best seed means trying all $2^S - 1$ of them; construction takes hundreds of nanoseconds per key. PHast⁺ uses instead the additive placement $p(s, c) = (c \bmod L) + s - 1$: the feasibility of 64 consecutive seeds of a bucket is the complement of the bitwise OR of 64-bit windows of the occupancy bitmap, one per key, and the best seed is the first feasible one. Construction becomes an order of magnitude faster, but with 8-bit seeds space grows from 1.92 to 2.12 bits per key; *wrapping* variants partially close the gap at the cost of slower construction.

This paper makes three contributions:

- an analysis of the space of PHast⁺ showing that its excess comes from bumping, that bumping is the slack the sweep cannot do without, and that a large part of it is caused by self-collisions inherent to additive placement (Section ??);

- PHast-R, a structure in which the seed chooses one of R independent offset patterns and a shift, and whose construction repairs failing buckets by evicting the owner of the only blocking slot, detected by a bit-parallel half adder (Section ??);
- an implementation in the Rust crate `sux` and an experimental comparison with the reference implementation of PHast and PHast⁺, together with a short account of the ideas that did not work (Sections ?? and ??).

2 The Space of PHast⁺

All measurements in this section refer to 10^7 random keys, 8-bit seeds, expected bucket size $\lambda = 5.25$ and slice length $L = 512$, the space-optimal configuration of PHast⁺. We use our own re-implementation, whose space matches the reference implementation within about 0.01 bits per key (the difference is the selection index of Elias–Fano).

Holes are bumped keys. The first level of PHast⁺ has output range $m = n$. Every key is either placed in a distinct slot or bumped, so the number of free slots (*holes*) is exactly the number of bumped keys. Each hole costs about $2 + \log_2(1/\beta)$ bits in the Elias–Fano remapping, where β is the fraction of bumped keys, and each bumped key must be placed again by the following levels. With S -bit seeds the space is thus approximately

$$\frac{S}{\lambda(1 - \beta)} + \beta \left(2 + \log_2 \frac{1}{\beta} \right),$$

and since the entropy of a random set of βn holes is already about $\beta n(\log_2(1/\beta) + \log_2 e)$, the encoding of holes cannot be substantially improved: *the only lever is β* . In our configuration $\beta = 7.6\%$, and bumping costs 0.58 of the 2.10 bits per key (0.46 for Elias–Fano, 0.12 for the seeds of further levels).

Bumping is necessary slack. One could try to reduce holes by overloading the first level ($m < n$), by retrying the seeds of whole blocks of buckets, or by giving failing buckets a second, wider seed. None of this works (Section ??). In particular, if failing buckets always succeed with a secondary seed, the fraction of buckets that fail with the primary seed grows from 6.8% to 46%: without bumped keys the region around the frontier of the sweep becomes completely full and every bucket faces a jam. Bumping keeps the frontier loose; the bump rate is the amount of slack that a placement scheme of given flexibility needs to flow, and the way to reduce it is to make placement more flexible.

Additive placement is rigid. Additive placement translates a fixed pattern: two keys of a bucket whose base positions coincide collide for every seed. For a bucket of size k this happens with probability about $k(k - 1)/(2L)$, so a fraction $\lambda^2/(2L) \approx 2.7\%$ of the buckets cannot be placed at all (we measure 2.6%); they are 38% of the bumped buckets and contain about $(\lambda^2 + 2\lambda)/(2L) \approx 3.7\%$ of the keys. Moreover, consecutive shifts are strongly correlated trials. Seeds, on the other hand, are almost incompressible: their empirical entropy is 7.73 bits, because the shift mostly encodes how far the leftmost key of the bucket lies behind the full part of its slice.

3 PHast-R

Placement. Each key has two 64-bit hashes, h and o . Let $R = 2^\rho$ be the number of *patterns*, $L = 2^\ell$ the slice length with $R\ell \leq 64$, and $D = \lfloor (2^S - 1)/R \rfloor$ the number of shifts. The bucket of a key is $\text{hi}(hB)$ and the start of its slice is $\sigma(h) = \text{hi}(h(m - L - D + 2))$, where $\text{hi}(x)$ denotes

the 64 most significant bits of a 128-bit product. The r -th pattern assigns to the key the in-slice offset

$$\omega_r(o) = \lfloor o/2^{r\ell} \rfloor \bmod L,$$

that is, the r -th group of ℓ bits of o : different patterns are independent. A seed $s \in [1 \dots RD]$ encodes the pair $r = (s - 1) \bmod R$, $d = \lfloor (s - 1)/R \rfloor$, and the key is mapped to

$$\sigma(h) + \omega_r(o) + d.$$

As in PHast, seed 0 means that the bucket is bumped. A query computes the same quantities as in PHast⁺ plus a shift and a mask; with $R = 1$ and $o = h$, PHast-R is exactly PHast⁺.

Search. For each pattern, the best shift is found as in PHast⁺: the occupancy words starting at the base positions $\sigma(h_i) + \omega_r(o_i)$ of the k keys are Ored, and the first zero bit is the smallest feasible shift; for $S = 8$ and $R = 4$ we have $D = 63$, so one word per key suffices. Among patterns, we keep the one minimizing the sum of positions, skipping a pattern as soon as the sum of its bases exceeds the best sum found. Since all R patterns of a bucket collide only with probability about $(k(k - 1)/2L)^R$, self-collisions disappear for $R \geq 2$, and independent patterns decorrelate the trials.

Repair. When no pattern admits a feasible shift, PHast-R tries to make room. Using the same words $w_1, \dots, w_k$, the half adder

$$t \leftarrow t \vee (u \wedge w_i), \quad u \leftarrow u \vee w_i$$

computes in $u \wedge \neg t$ the shifts at which *exactly one* key is blocked. For each such candidate, in order of increasing sum of positions, an *owner* array parallel to the cyclic occupancy bitmap gives the bucket Y occupying the blocking slot. If Y is still entirely inside the active part of the window, we evict Y , place the current bucket, and place Y again; if Y fails, it may in turn evict another bucket (with a quarter of the candidates), up to a maximum depth. If Y cannot be placed, the eviction is undone and the next candidate (we keep one per distinct Y) is tried. A failing bucket is bumped only after 16 unsuccessful candidates. Repair turns the greedy sweep of PHast into a local search, as in PtrHash [?], but it is triggered only by failures, and bumping remains as a safety valve: the structure stays minimal and needs no slack in the output range.

Levels. Keys of bumped buckets are mapped to a new pair of hashes by a mixing function of (h, o) , so construction never rehashes keys and a query for a bumped key needs no further hashing of the key. Levels after the first use the same scheme with range equal to their number of keys; when fewer than 4096 keys are left, a last level uses a slightly larger range and no bumping, retrying with a different salt in the unlikely case of failure. As in PHast⁺, an Elias–Fano sequence maps the outputs of the levels after the first to the holes of the first level.

Parallel construction. We adopt the scheme of PHast: buckets are split into chunks separated by gaps of $\lceil (L + D)B/m \rceil + 1$ buckets, so that the slices of different chunks cannot overlap. Chunks are swept in parallel; then gaps are swept in parallel with the slots of the neighboring buckets marked as occupied and not evictable.

4 Experiments

We ran all experiments on an Apple M1 Max with 64 GiB of RAM, compiling with Rust 1.98 at optimization level 3 with thin link-time optimization. Keys are distinct 64-bit integers. The reference implementation is the `ph` crate by Beling (Function2 with the `ShiftOnly`,

Table 1: Comparison on 10^7 keys (single-threaded construction). d is the maximum depth of repair ($d = 0$: no repair).

Method	Configuration	bits/key	build [ns/key]	query [ns]
PHast ⁺	$S = 8, \lambda = 5.25$	2.116	40	22.8
PHast ⁺ wrap $\delta = 1$	$S = 8, \lambda = 5.25$	2.095	48	22.0
PHast ⁺ wrap $\delta = 2$	$S = 8, \lambda = 5$	2.011	59	20.9
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.970	70	21.3
PHast	$S = 8, \lambda = 4.5$	1.923	562	19.9
PHast-R	$S = 8, L = 512, \lambda = 5, d = 0$	2.022	48	22.2
PHast-R	$S = 8, L = 512, \lambda = 5, d = 1$	1.935	79	21.4
PHast-R	$S = 8, L = 512, \lambda = 5, d = 2$	1.924	127	21.4
PHast ⁺	$S = 10, \lambda = 5.15$	2.085	38	19.7
PHast ⁺ wrap $\delta = 1$	$S = 10, \lambda = 6.2$	1.906	60	19.9
PHast ⁺ wrap $\delta = 2$	$S = 10, \lambda = 5.9$	1.884	87	19.7
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.872	121	20.1
PHast	$S = 10, \lambda = 6.05$	1.854	1646	19.9
PHast-R	$S = 9, L = 1024, \lambda = 5.75, d = 1$	1.888	85	24.8
PHast-R	$S = 10, L = 2048, \lambda = 6, d = 0$	1.922	52	24.2
PHast-R	$S = 10, L = 2048, \lambda = 6, d = 1$	1.861	86	23.7
PHast-R	$S = 10, L = 2048, \lambda = 6.25, d = 2$	1.854	137	24.0
PHast-R	$S = 11, L = 4096, \lambda = 6.75, d = 1$	1.860	88	24.1

Table 2: Comparison on 10^8 keys, with single-threaded and 10-threaded construction.

Method	Configuration	bits/key	build [ns/key]		query [ns]
			1 thread	10 threads	
PHast ⁺	$S = 8, \lambda = 5.25$	2.115	44	9.1	38.6
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.968	75	13.6	34.9
PHast	$S = 8, \lambda = 4.5$	1.921	575	–	31.1
PHast-R	$S = 8, \lambda = 5, d = 1$	1.934	81	13.8	34.9
PHast-R	$S = 8, \lambda = 5, d = 2$	1.923	127	18.7	35.0
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.869	126	19.2	32.5
PHast-R	$S = 10, \lambda = 6, d = 1$	1.857	87	13.0	37.3
PHast-R	$S = 10, \lambda = 6.25, d = 2$	1.850	137	19.4	38.3

ShiftOnlyWrapped $\langle\delta\rangle$ and **SeedOnly** seed choosers for PHast⁺, PHast⁺ with wrapping and PHast, respectively). To make query times comparable we fed both implementations the same hash function, XXH3-64 with a seed. Construction times are single-threaded unless otherwise noted and include hashing; query times are the best of three averages over 10^7 queries of random keys of the set. PHast-R uses $R = 4$ patterns, 16 repair candidates, and byte seeds for $S = 8$ and bit-packed seeds otherwise.

Table ?? and Figure ?? show the results on 10^7 keys. Patterns alone (PHast-R with $d = 0$) reduce space by 0.09 bits per key with respect to PHast⁺ at a cost of 7 ns per key. Repair is worth another 0.06–0.10 bits per key. With 8-bit seeds and depth 2, PHast-R uses the same space as PHast and builds 4.4 times faster; with 10-bit seeds and depth 2 it matches PHast in space and builds 12 times faster. Against the best PHast⁺ variant with wrapping, PHast-R with depth 1 uses 0.035 fewer bits per key with 8-bit seeds at 13% more construction time, and with 10-bit seeds it is both smaller and 29% faster to build. Queries with byte seeds are as fast as in PHast⁺; bit-packed seeds currently cost about 4 ns more in our implementation.

Table ?? shows that the picture does not change on 10^8 keys, where queries are dominated

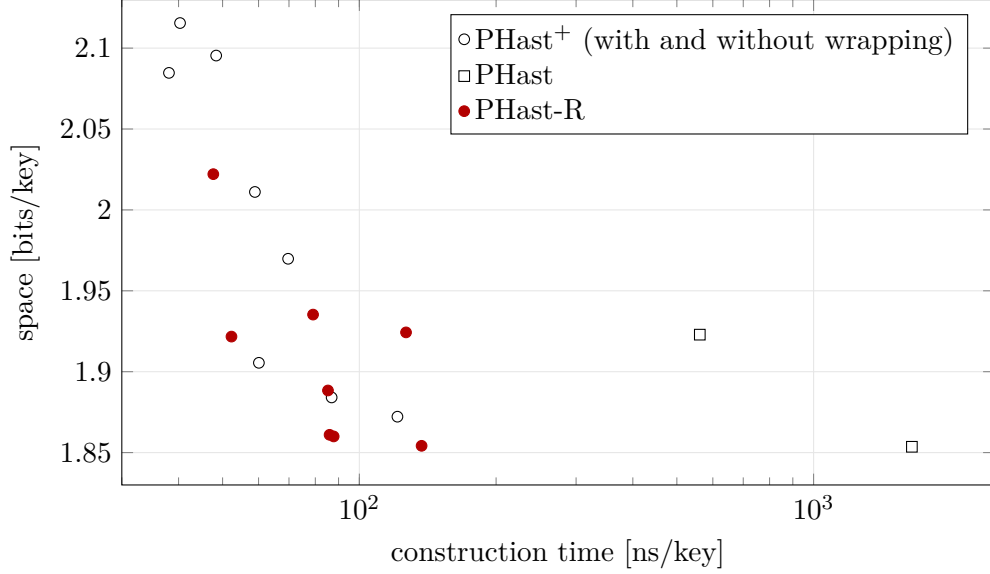


Figure 1: Space versus single-threaded construction time on 10^7 keys (data of Table ??).

by cache misses. With 10 threads (8 performance and 2 efficiency cores), PHast-R with depth 1 builds about as fast as PHast⁺ with wrapping and 8-bit seeds, and faster with 10-bit seeds: repair is local, so it parallelizes as well as the rest of the sweep.

5 What Did Not Work

Table ?? lists the alternatives we evaluated before PHast-R, all on 10^7 keys with 8-bit seeds (baseline: PHast⁺ with $\lambda = 5.25$, 2.105 bits per key in our implementation). They share a lesson: holes are the price of the slack that the sweep needs, so any idea that removes slack without adding flexibility makes things worse.

6 Conclusions

The space of PHast⁺ is dominated by the cost of its slack, and slack is what a greedy sweep with rigid additive placement needs. Independent offset patterns remove the rigidity without losing bit-parallelism, and a half adder over the same words that test feasibility turns failures into cheap, local cuckoo repairs. The resulting structure, PHast-R, has the query of PHast⁺, the space of PHast, and an intermediate construction time. Natural next steps are retuning the bucket priority for larger seeds, faster bit-packed seed access, and replacing the last levels with a more compact structure.

The implementation is available in the `sux` crate as `sux::func::PHastR`.

References

- [1] P. Beling and P. Sanders. PHast – Perfect Hashing made fast. arXiv:2504.17918, 2025.
- [2] R. Groot Koerkamp. PtrHash: Minimal perfect hashing at RAM throughput. In *23rd International Symposium on Experimental Algorithms (SEA)*, LIPIcs 338, pages 21:1–21:21, 2025.
- [3] P. Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM*, 21(2):246–260, 1974.

Table 3: Ideas that did not improve on PHast⁺ (10^7 keys, $S = 8$; best configuration found for each idea).

Idea	bits/key
Overloaded first level ($m = 0.96n$): holes only drop from 7.6% to 6.3%	2.108
Underloaded first level ($m = 1.05n$)	2.160
Block reseeding: an extra seed per 256 buckets, best of 16 retries	2.061
Mixed bucket sizes (three buckets with $\lambda = 6$, one with $\lambda = 2$)	2.196
A second sweep of 10% “filler” keys in tiny buckets with their own seeds	2.595
Two-tier seeds: an escape value and a 12-bit secondary seed (jam: escapes grow to 46% of the buckets)	2.694
Chained seeds: the pattern of a bucket depends on the seed of the previous one, with backtracking (requires index order)	2.150
Partial bumping by fingerprint thresholds encoded in the seed (1.93 if the threshold were free)	2.107
Steering holes to a lattice of slots, to encode them cheaply (holes on the lattice grow from 1.0% to at most 4.0% of the slots)	2.104
Row-permutation placement: 64 column permutations tested in parallel	2.057
k -perfect PHast ($k = 8$, 0.675 bits/key) followed by bijections	≥ 1.85
Cuckoo repair on PHast with pseudorandom placement (from 1.920)	1.916

- [4] R. M. Fano. On the number of bits required to implement an associative memory. Memorandum 61, Computer Structures Group, Project MAC, MIT, 1971.